Datos del estudiante

Nombre y apellidos	Mikel Itarte Nausía
Fecha de entrega	23/03/2026

Actividad 3. De la idea a la base de datos: gestión de un centro de eventos en MySQL

Objetivos de la actividad

Con esta actividad grupal, los estudiantes van a conseguir:

- **Aplicar** los principios del diseño conceptual y relacional de bases de datos a un escenario real.
- **Colaborar** en la creación de un esquema de base de datos relacional robusto y coherente.
- **Implementar** el diseño de una base de datos utilizando el lenguaje **SQL** en **MySQL Workbench** (o similar).
- **Insertar** datos de prueba de manera eficiente para poblar la base de datos.
- **Formular** y ejecutar **consultas SQL** de diferentes tipos para extraer información relevante de la base de datos.
- **Desarrollar** habilidades de trabajo en equipo y resolución de problemas en un entorno de desarrollo de bases de datos.

Pautas de elaboración

Esta práctica grupal se centra en el diseño e implementación de una base de datos para un **centro de eventos**, que organiza y gestiona diversos tipos de eventos (conciertos, conferencias, talleres) y a sus participantes.

Organización del grupo: esta actividad está diseñada para ser realizada en grupos de 3 a 4 estudiantes. Se recomienda que cada miembro del grupo asuma un rol (ej.: diseñador conceptual, implementador SQL, gestor de datos, creador de consultas), aunque la responsabilidad final es compartida.

Aquí te explicamos cómo proceder, paso a paso:

1. **Análisis de requisitos y diseño conceptual (Fase 1: 30 % del tiempo estimado):**

- **Lectura del escenario:** el centro de eventos necesita gestionar:
    - **Eventos:** cada evento tiene un título, fecha, hora de inicio, hora de fin, descripción, aforo máximo y tipo de evento (ej. «Concierto», «Conferencia», «Taller»).
    - **Salas:** el centro tiene varias salas, cada una con un nombre, ubicación y capacidad. Cada evento se realiza en una única sala.
    - **Participantes:** los participantes se registran con su nombre, apellidos, email y teléfono. Un participante puede inscribirse en varios eventos.
    - **Inscripciones:** es necesario registrar qué participante se inscribe en qué evento y la fecha de su inscripción.
    - **Organizadores:** algunos eventos pueden tener uno o varios organizadores (personas o entidades). Se necesita registrar el nombre del organizador y su contacto. Un organizador puede organizar varios eventos y un evento puede tener varios organizadores.
- **Identificación E/R:** en grupo, identifiquen las **entidades** principales, sus **atributos** (incluyendo claves primarias) y las **relaciones** entre ellas (con su cardinalidad N:M, 1:N, 1:1) y posibles atributos de las relaciones.
- **Paso a modelo relacional:** a partir de su diseño E/R, transformen este modelo conceptual en un **modelo relacional**. Definan las tablas, sus columnas, tipos de datos apropiados, claves primarias (PK) y claves foráneas (FK), indicando a qué tabla referencian. Consideren la normalización para evitar redundancias obvias. **No es necesario dibujar el DER, solo describir el modelo relacional resultante.**

2. **Creación de la base de datos y tablas en MySQL (Fase 2: 30 % del tiempo estimado):**
   - **Conexión:** utilicen MySQL Workbench o la línea de comandos para conectarse a un servidor MySQL.
   - **Creación de la base de datos:** creen una base de datos con un nombre significativo (ej.: centro_eventos_grupoX).
   - **Creación de tablas:** escriban y ejecuten las sentencias **CREATE TABLE** para cada una de las tablas definidas en el paso anterior. Asegúrense de incluir las claves primarias y las claves foráneas, con sus respectivas restricciones (ej.: NOT NULL, UNIQUE, DEFAULT).

3. **Inserción de datos de prueba (Fase 3: 20 % del tiempo estimado)**
   - **Población de tablas:** escriban y ejecuten sentencias **INSERT INTO** para poblar cada tabla con al menos **cinco registros significativos**. Asegúrense de que los datos sean coherentes con las relaciones establecidas (por ejemplo, no pueden insertar una inscripción para un evento que no existe). Los datos deben simular un escenario real y permitir probar las consultas posteriores.

4. **Creación de consultas SQL (Fase 4: 20 % del tiempo estimado):** diseñen y ejecuten **cuatro consultas SQL diferentes** sobre su base de datos. Cada consulta debe ser de un tipo distinto y significativa para el escenario del centro de eventos:
   - **Consulta 1: selección simple con filtro (SELECT con WHERE)**. Ejemplo: «Obtener todos los eventos de tipo “Concierto” en el mes de octubre».
   - **Consulta 2: consulta con JOIN y ordenación (SELECT con JOIN y ORDER BY)**. Ejemplo: «Listar el nombre de todos los participantes y los títulos de los eventos en los que se han inscrito, ordenados por fecha de inscripción».
   - **Consulta 3: consulta de agregación (SELECT con GROUP BY y función de agregación)**. Ejemplo: «Contar cuántos eventos se han realizado en cada tipo de sala y mostrar el nombre de la sala y el número de eventos».
   - **Consulta 4: consulta avanzada (Subconsulta o JOIN Múltiple o con HAVING)**. Ejemplo: «Encontrar los participantes que se han inscrito en más de dos eventos».

5. **Entrega:** se entregará un único documento PDF por grupo que contenga:
   - La descripción de su **modelo relacional** (nombre de tablas, columnas, PK, FK).
   - Las sentencias **CREATE TABLE** utilizadas.
   - Las sentencias **INSERT INTO** utilizadas.
   - Las **cuatro consultas SQL** desarrolladas, junto con una breve descripción de lo que pretende cada consulta y el resultado obtenido.

Extensión y formato

Extensión máxima: 10 páginas (incluyendo código SQL y resultados).

Formato: documento PDF.

Fuente: consola (ej. Courier New, Consolas) para el código SQL y Arial o Calibri para el texto, tamaño 10 para código, tamaño 11 para texto.

Interlineado: 1,0 para código, 1,5 para texto.

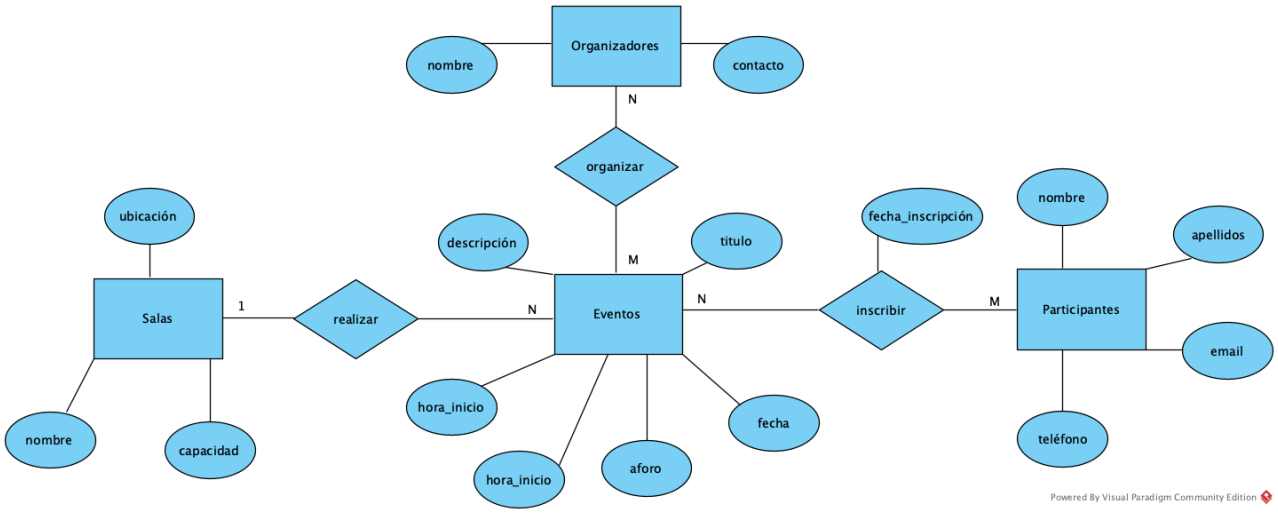
Criterios de evaluación

Título de la actividad	Descripción	Peso %
Diseño relacional	La transformación del escenario a un modelo relacional es correcta, lógica y coherente. Se identifican correctamente tablas, columnas, PK y FK, aplicando buenas prácticas de normalización básica.	25%
Implementación en MySQL	Las sentencias CREATE TABLE son sintácticamente correctas, ejecutan sin errores y reflejan fielmente el diseño relacional. Se manejan correctamente los tipos de datos y las restricciones de integridad.	25%
Inserción de datos	Las sentencias `INSERT INTO` son correctas y los datos de prueba son suficientes (al menos cinco por tabla), variados y coherentes, permitiendo probar las consultas.	20%
Consultas SQL	Se presentan 4 consultas de distinto tipo. Cada consulta es sintácticamente correcta, resuelve el problema planteado, y demuestra una comprensión adecuada de la lógica de consulta SQL (JOINs, agregación, subconsultas/HAVING, filtrado).	20%
Trabajo en equipo y presentación	La colaboración grupal es evidente en el resultado final. El documento es claro, bien estructurado, cumple con el formato y presenta la información de manera ordenada y profesional.	10%
		100 %

Composición del equipo y reparto de tareas:

NOMBRE	SECCIONES TRABAJADAS
Iker Delgado López	Identificación E/R y paso a modelo relacional.
Sthefania Avila Avila	Creación de la BBDD y las tablas correspondientes.
Boris Yepez Zamora	Inserción de los datos de prueba
Mikel Itarte Nausía	Generar consultas y documentación

Eventos Modelo E/R



Modelo Relacional

Salas

Columna	Tipo de dato	Restricciones
id_sala	INT AUTO_INCREMENT	PK
nombre	VARCHAR(100)	NOT NULL
ubicacion	VARCHAR(100)	
capacidad	INT	NOT NULL

Eventos

Columna	Tipo de dato	Restricciones
id_evento	INT AUTO_INCREMENT	PK
titulo	VARCHAR(150)	NOT NULL
fecha	DATE	
hora_inicio	TIME	
hora_fin	TIME	
descripción	TEXT	
aforo_maximo	INT	
tipo_evento	VARCHAR(50)	
id_sala	INT	FK -> Salas(id_sala)

Participantes

Columna	Tipo de dato	Restricciones
id_participante	INT AUTO_INCREMENT	PK
nombre	VARCHAR(100)	
apellidos	VARCHAR(100)	
email	VARCHAR(100)	UNIQUE
telefono	VARCHAR(20)	

Inscripciones

Columna	Tipo de dato	Restricciones
id_inscripcion	INT AUTO_INCREMENT	PK
id_participante	INT	FK -> Participantes(id_participante)
id_evento	INT	FK -> Eventos(id_evento)
fecha_inscripcion	DATE	

Organizadores

Columna	Tipo de dato	Restricciones
id_organizador	INT AUTO_INCREMENT	PK
nombre	VARCHAR(100)	
contacto	VARCHAR(100)	

Evento_Organizador

Columna	Tipo de dato	Restricciones
id_evento	INT	FK -> Eventos(id_evento)
id_organizador	INT	FK -> Organizadores(id_organizador)

Creación de la BBDD y las sentencias:

```sql
CREATE DATABASE IF NOT EXISTS centro_eventos_grupoX;
USE centro_eventos_grupoX;
/*creamos tabla salas */

CREATE TABLE IF NOT EXISTS Salas  (
    id_sala INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    ubicacion VARCHAR(100),
    capacidad INT NOT NULL
);

/* creamos la tabla participantes */

CREATE TABLE IF NOT EXISTS Participantes (
    id_participante INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100),
    apellidos VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    telefono VARCHAR(20)
);
```

```sql
CREATE TABLE IF NOT EXISTS Organizadores (
    id_organizador INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100),
    contacto VARCHAR(100)
    );


CREATE TABLE IF NOT EXISTS Eventos (
    id_evento INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    fecha DATE,
    hora_inicio TIME,
    hora_fin TIME,
    descripcion TEXT,
    aforo_maximo INT,
    tipo_evento VARCHAR(50),
    id_sala INT,
    CONSTRAINT fk_evento_sala FOREIGN KEY (id_sala) REFERENCES Salas(id_sala)
);
```

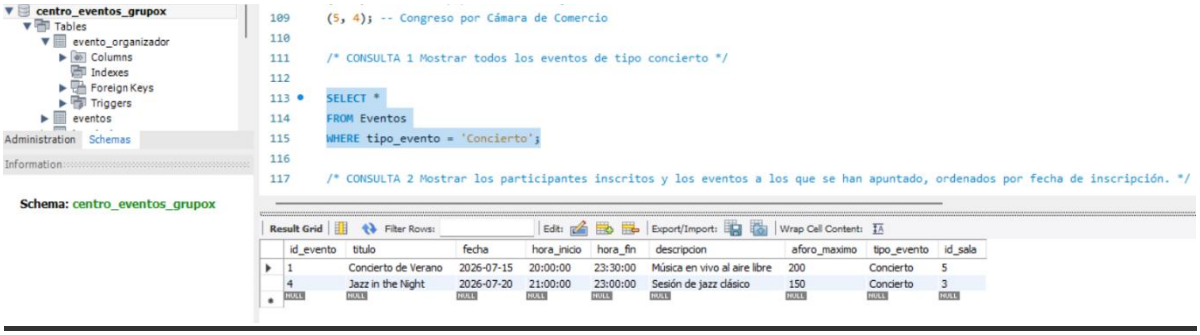
```sql
CREATE TABLE IF NOT EXISTS Inscripciones (
    id_inscripcion INT AUTO_INCREMENT PRIMARY KEY,
    id_participante INT,
    id_evento INT,
    fecha_inscripcion DATE,
    CONSTRAINT fk_inscripcion_participante FOREIGN KEY (id_participante) REFERENCES Participantes(id_participante),
    CONSTRAINT fk_inscripcion_evento FOREIGN KEY (id_evento) REFERENCES Eventos(id_evento)
);

CREATE TABLE IF NOT EXISTS Evento_Organizador (
    id_evento INT,
    id_organizador INT,
    PRIMARY KEY (id_evento, id_organizador), -- Clave primaria compuesta
    CONSTRAINT fk_eo_evento FOREIGN KEY (id_evento) REFERENCES Eventos(id_evento),
    CONSTRAINT fk_eo_organizador FOREIGN KEY (id_organizador) REFERENCES Organizadores(id_organizador)
);
```

Inserción de los datos:

```sql
/* Poblamos la tabla Salas */
INSERT INTO Salas (nombre, ubicacion, capacidad) VALUES
('Gran Auditorio', 'Planta Baja', 500),
('Sala de Juntas B', 'Planta 1', 50),
('Anfiteatro Norte', 'Planta 2', 150),
('Laboratorio Tech', 'Sótano', 30),
('Espacio Abierto', 'Jardín Exterior', 200);

/* Poblamos la tabla Organizadores */
INSERT INTO Organizadores (nombre, contacto) VALUES
('Eventos Globales S.A.', 'contacto@eventosglobales.com'),
('Asociación de Músicos', 'info@musicos.org'),
('Tech Academy', 'admin@techacademy.es'),
('Cámara de Comercio', 'secretaria@camara.com'),
('Ayuntamiento - Cultura', 'cultura@ayuntamiento.gob');

/* Poblamos la tabla Eventos (Referenciando a Salas) */
INSERT INTO Eventos (titulo, fecha, hora_inicio, hora_fin, descripcion, aforo_maximo, tipo_evento, id_sala) VALUES
('Concierto de Verano', '2026-07-15', '20:00:00', '23:30:00', 'Música en vivo al aire libre', 200, 'Concierto', 5),
('Simposio de IA', '2026-08-10', '09:00:00', '14:00:00', 'Novedades en Inteligencia Artificial', 50, 'Conferencia', 2),
('Workshop SQL Avanzado', '2026-09-05', '16:00:00', '20:00:00', 'Taller práctico de bases de datos', 30, 'Taller', 4),
('Jazz in the Night', '2026-07-20', '21:00:00', '23:00:00', 'Sesión de jazz clásico', 150, 'Concierto', 3),
('Congreso de Medicina', '2026-10-12', '08:30:00', '18:00:00', 'Avances en cardiología', 400, 'Conferencia', 1);

/* Poblamos la tabla Participantes */
INSERT INTO Participantes (nombre, apellidos, email, telefono) VALUES
('Carlos', 'Ruiz Zafón', 'carlos.ruiz@mail.com', '600111222'),
('Laura', 'García Lorca', 'laura.garcia@mail.com', '600333444'),
('Miguel', 'de Cervantes', 'miguel.cervantes@mail.com', '600555666'),
('Elena', 'Ferrante', 'elena.ferrante@mail.com', '600777888'),
('Isabel', 'Allende', 'isabel.allende@mail.com', '600999000');
```

```sql
/* Poblamos la tabla Inscripciones (Relación Participante-Evento) */
INSERT INTO Inscripciones (id_participante, id_evento, fecha_inscripcion) VALUES
(1, 1, '2026-06-01'), -- Carlos en Concierto
(1, 3, '2026-06-05'), -- Carlos en Workshop (Más de un evento)
(2, 2, '2026-07-10'), -- Laura en Simposio
(3, 4, '2026-07-12'), -- Miguel en Jazz
(4, 5, '2026-08-01'), -- Elena en Congreso
(5, 1, '2026-06-15'), -- Isabel en Concierto
(5, 2, '2026-06-20'); -- Isabel en Simposio (Más de un evento)

/* Poblamos la tabla Evento_Organizador */
INSERT INTO Evento_Organizador (id_evento, id_organizador) VALUES
(1, 2), -- Concierto organizado por Asociación de Músicos
(1, 5), -- Concierto también organizado por Ayuntamiento
(2, 3), -- Simposio por Tech Academy
(3, 3), -- Workshop por Tech Academy
(5, 4); -- Congreso por Cámara de Comercio
```

Generar consultas:




```sql
/* CONSULTA 2 Mostrar los participantes inscritos y los eventos a los que se han apuntado, ordenados por fecha de inscripción. */

SELECT
    p.nombre,
    p.apellidos,
    e.titulo,
    i.fecha_inscripcion
FROM Inscripciones i
JOIN Participantes p ON i.id_participante = p.id_participante
JOIN Eventos e ON i.id_evento = e.id_evento
ORDER BY i.fecha_inscripcion ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nombre	apellidos	titulo	fecha_inscripcion
Carlos	Ruiz Zafón	Concierto de Verano	2026-06-01
Carlos	Ruiz Zafón	Workshop SQL Avanzado	2026-06-05
Isabel	Allende	Concierto de Verano	2026-06-15
Isabel	Allende	Simposio de IA	2026-06-20
Laura	García Lorca	Simposio de IA	2026-07-10
Miguel	de Cervantes	Jazz in the Night	2026-07-12
Elena	Ferrante	Congreso de Medicina	2026-08-01

```sql
/* CONSULTA 3 Contar el número de eventos asignados a cada sala. */

SELECT
    s.nombre AS sala,
    COUNT(e.id_evento) AS numero_eventos
FROM Salas s
JOIN Eventos e ON s.id_sala = e.id_sala
GROUP BY s.id_sala, s.nombre;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

sala	numero_eventos
Gran Auditorio	1
Sala de Juntas B	1
Anfiteatro Norte	1
Laboratorio Tech	1
Espacio Abierto	1

```sql
/* CONSULTA 4 Mostrar los participantes inscritos en más de un evento. */

SELECT
    p.nombre,
    p.apellidos,
    COUNT(i.id_evento) AS total_eventos
FROM Participantes p
JOIN Inscripciones i ON p.id_participante = i.id_participante
GROUP BY p.id_participante, p.nombre, p.apellidos
HAVING COUNT(i.id_evento) > 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nombre	apellidos	total_eventos
Carlos	Ruiz Zafón	2
Isabel	Allende	2